HeyNeighbor | Sprint Cycle III

Database Implementation

Community gift, trade and resource-sharing platform

Trinity Banks, Christopher Miller, Kenyce Holloman & Karima Dewdney



PostgreSQL on Neon

1 Relational fit

20 related tables with primary keys, foreign keys, NOT NULL and CHECK constraints.

2 Rich constraints

UNIQUE, composite keys and CHECK rules enforce many business rules in the database.

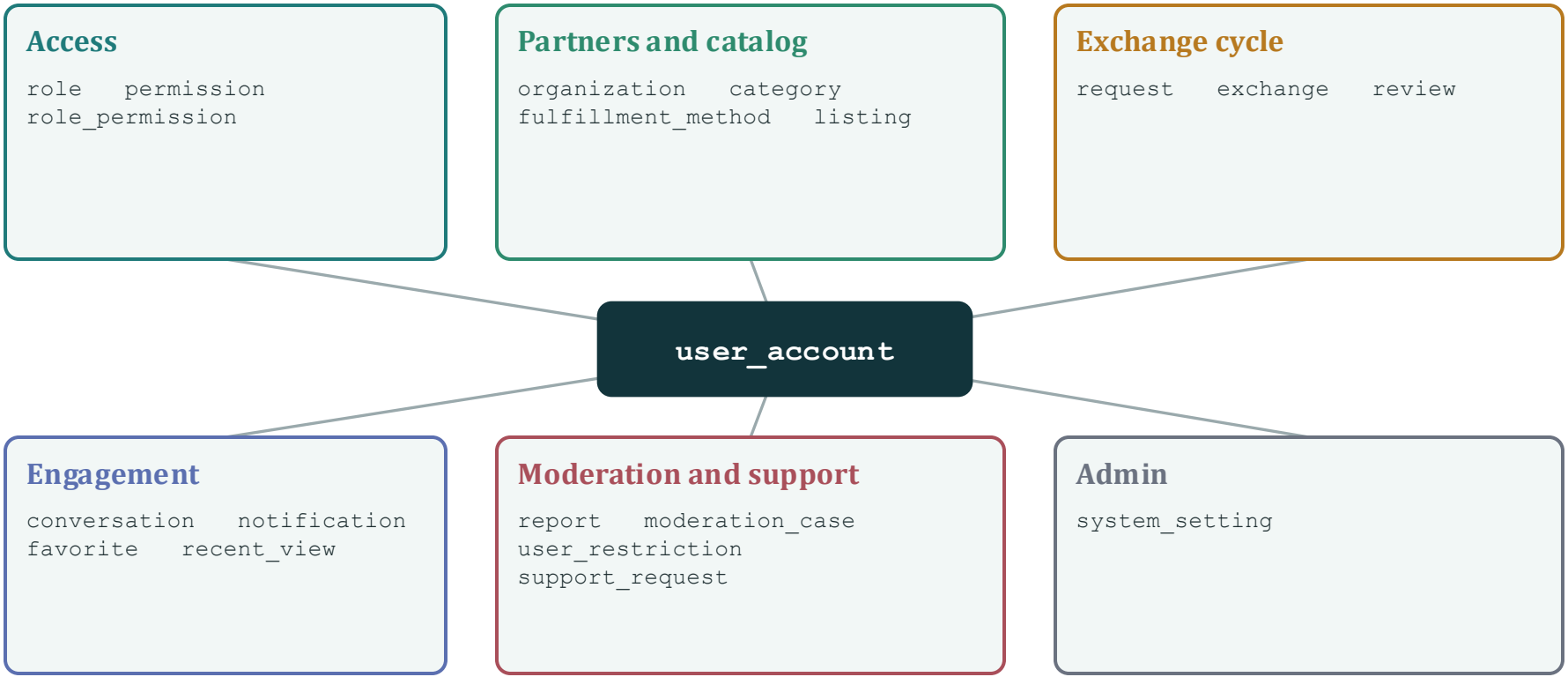
3 JSONB columns

Messages, participants, notes and listing images stay as queryable lists.

4 Neon platform

Serverless Postgres with a browser SQL Editor, branching and a free tier.

Updated ER diagram: 20 tables



Almost every table links to user_account (owner, requester, reviewer, moderator)

Design review: 20 tables

Removed

- audit_log (status changes live on user_account)
- organization_hours (now operating_hours text)
- listing_status and listing_category tables
- escalation table

Merged

- user_profile into user_account
- dispute and support_note into support_request
- trade_proposal into request
- listing_photo into listing
- message and participants into conversation
- moderation_action into moderation_case

Kept and added

- user_restriction stays its own table
- Role change uses a support_request of type role_change, approved only by an administrator

Implemented database structure

20

tables

12

tables with data

- Identity primary keys and foreign keys on every relationship
- CHECK constraints: statuses, 1 to 5 ratings, ends_at after starts_at
- UNIQUE: email, role_name, setting_key, organization owner
- JSONB for messages, notes and participants

Who built which tables

Access and partners (Karima Dewdney)
`role, permission, role_permission, user_account, organization`

Catalog and exchange (Kenyce Holloman)
`category, fulfillment_method, listing, request, exchange`

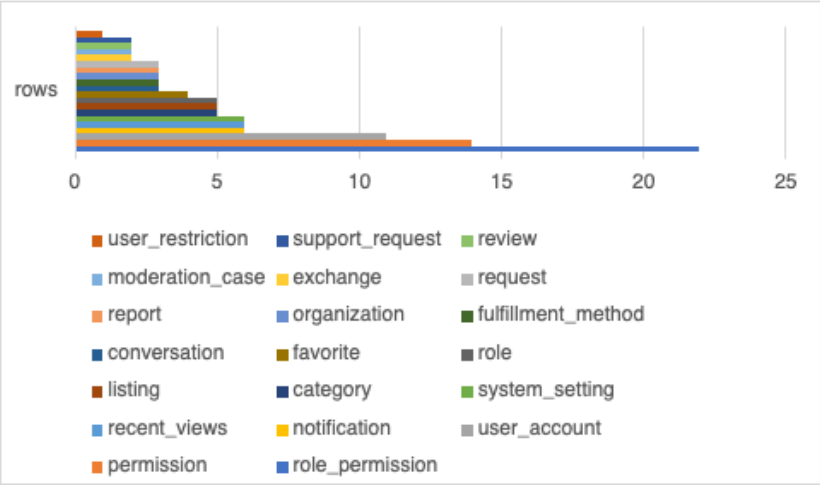
Moderation and support (Christopher Miller)
`support_request, conversation, moderation_case, report, user_restriction`

Reviews and settings (Trinity Banks)
`review, favorite, recent_view, notification, system_setting`

Meaningful sample data

One story across the tables

category_id integer	listing_status varchar(2…	fulfillment_method_id i…	title varchar(150)	description text	listing_type varchar(10)
1	completed	1	Homemade banana bread …	Two fresh loaves, bake…	gift
1	active	1	Holiday meal boxes	Boxes with turkey, sid…	gift
	active	3	Free math tutoring …	One-hour sessions cove…	gift
4	active	1	Bike tune-up and repair	Brakes, gears and tire…	trade
5	active	1	Kids winter coats, siz…	Clean coats, gently us…	gift



Row per table

Tools, scripts and run order

1

Create table Scripts

Parent tables first: role, user_account, listing, exchange

2

Insert sample data

Roles first, then users, listings, requests, exchanges

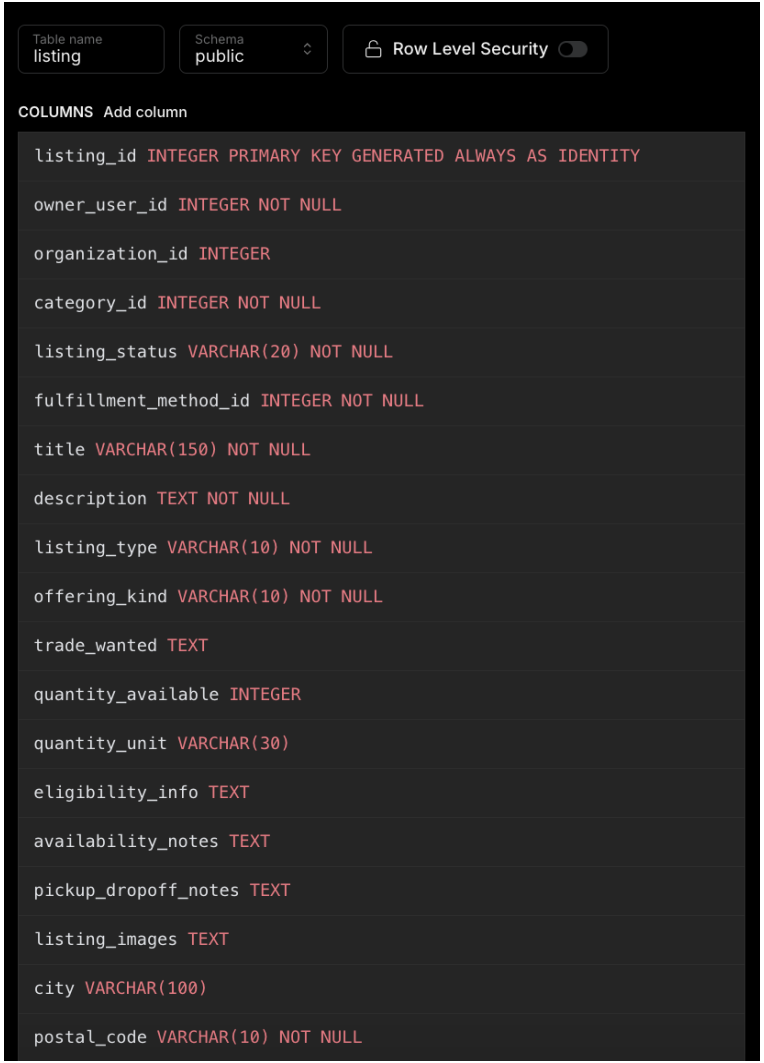
3

Queries and checks

Search, related data, add, modify, constraint tests

Native tools

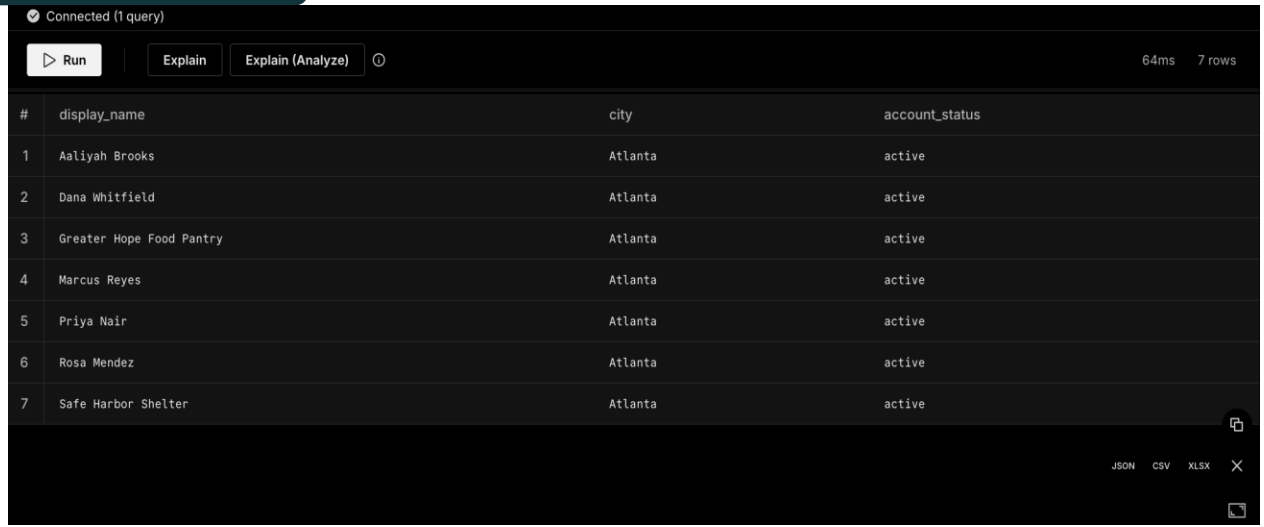
- Neon SQL Editor
- information_schema and pg_constraint to inspect the structure



Demo: search data

A. Search: active users in Atlanta

```
SELECT display_name, city, account_status FROM
user_account WHERE city = 'Atlanta' AND account_status =
'active' ORDER BY display_name;
```

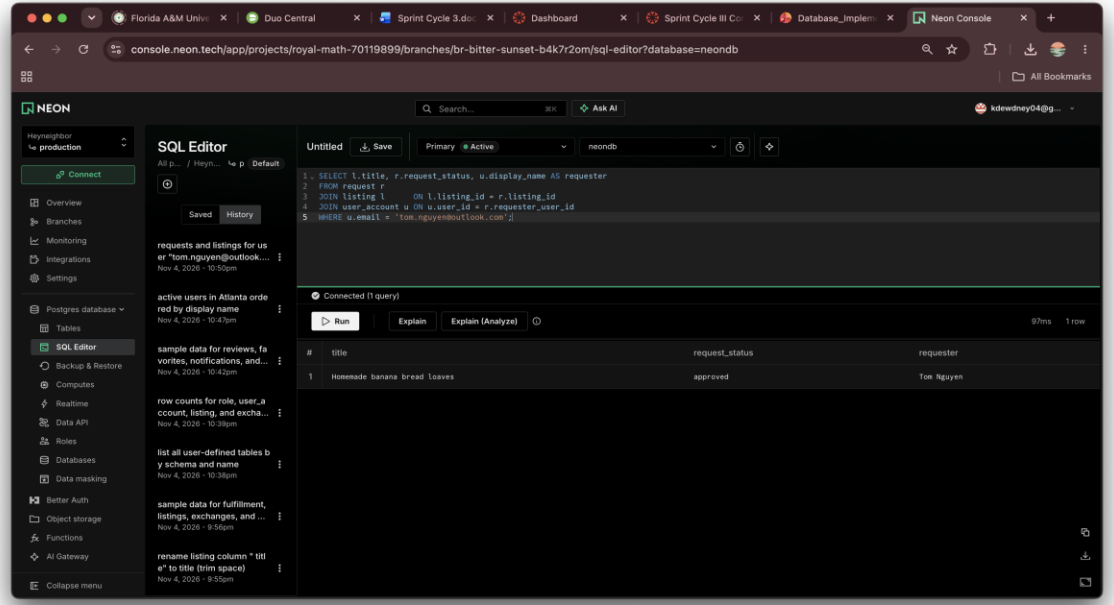


#	display_name	city	account_status
1	Aaliyah Brooks	Atlanta	active
2	Dana Whitfield	Atlanta	active
3	Greater Hope Food Pantry	Atlanta	active
4	Marcus Reyes	Atlanta	active
5	Priya Nair	Atlanta	active
6	Rosa Mendez	Atlanta	active
7	Safe Harbor Shelter	Atlanta	active

Demo: Related data

B. Related data: one user’s requests

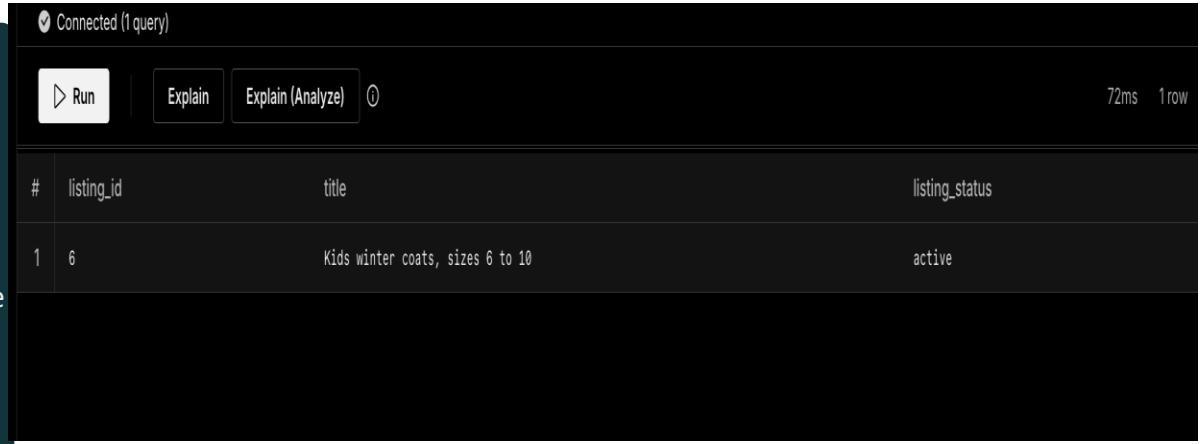
```
SELECT l.title, r.request_status, u.display_name AS
requester FROM request r JOIN listing l ON l.listing_id =
r.listing_id JOIN user_account u ON u.user_id =
r.requester_user_id WHERE u.email =
'tom.nguyen@outlook.com';
```



Demo: Add data

C. Add: a new user signs up

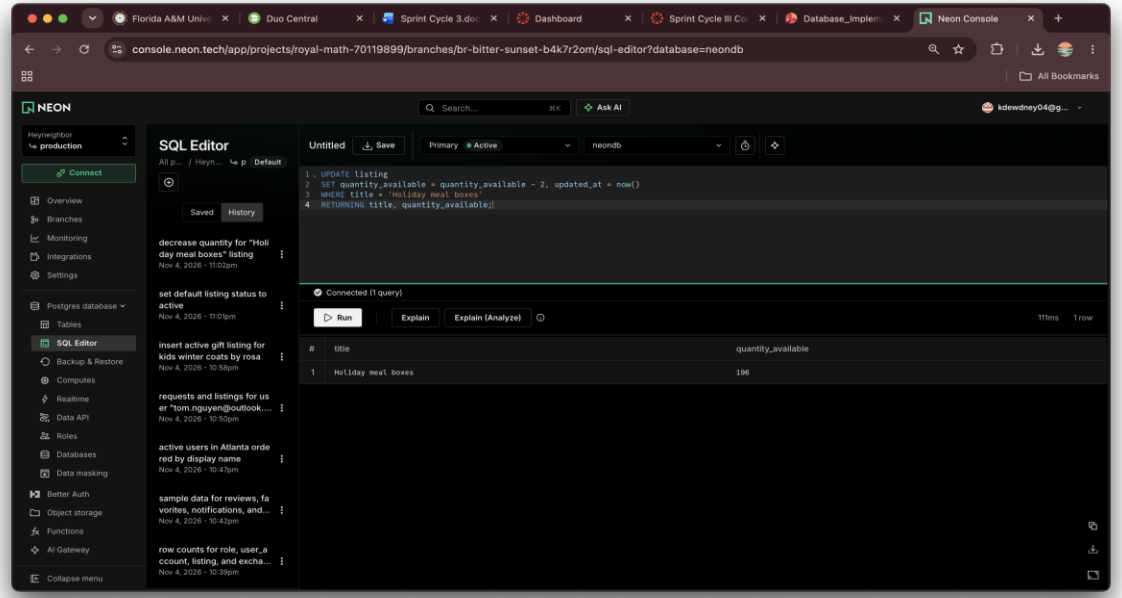
```
INSERT INTO listing (owner_user_id, category_id,
fulfillment_method_id, listing_status, title,
description, listing_type, offering_kind,
postal_code, latitude, longitude, published_at)
VALUES ((SELECT user_id FROM user_account
WHERE email = 'rosa.mendez@yahoo.com'),
(SELECT category_id FROM category WHERE name
= 'Household items'), (SELECT method_id FROM
fulfillment_method WHERE method_name =
'pickup'), 'active', 'Kids winter coats, sizes 6 to 10',
'Clean coats, gently used', 'gift', 'good', '30318',
33.791100, -84.447000, now()) RETURNING
listing_id, title, listing_status;
```



Demo: Modify data

D. Modify: reduce a listing's quantity

```
UPDATE listing SET quantity_available = quantity_available - 2, updated_at = now() WHERE title = 'Holiday meal boxes' RETURNING title, quantity_available;
```



Three use cases the database supports

1

Gift: listing, request, exchange

- listing, request, exchange, fulfillment_method
- Banana bread: Aaliyah offers, Tom requests
- exchange.request_id is unique
- status ends as completed

2

No-show dispute

- exchange marked disputed
- support_request type dispute, dispute_type no_show
- related_exchange_id required by CHECK
- assigned coordinator and JSONB notes

3

Partner approval

- user_account with role community_partner
- organization: one per owner
- approval_status pending to approved
- reviewed_by and reviewed_at recorded

Database rules and how they are enforced

Rule	Enforced by	How
Br-01 Email is unique	Database	UNIQUE on user_account.email
Br-10 One organization per owner	Database	UNIQUE on organization.owner_user_id
Br-45 One review per exchange and reviewer	Database	UNIQUE (exchange_id, reviewer_user_id)
Br-46 Ratings are 1 to 5	Database	CHECK constraint; a rating of 6 is rejected
Br-66 Dispute needs an exchange	Database	CHECK on support_request
Br-73 Admin-only permissions	Application	Checked on grant in Sprint IV
Br-76 System roles not deletable	Application	Delete refuses is_system_role

GitHub, team and wrap-up

GitHub repository

- Schema and create scripts
- Sample-data scripts
- Query file with rule tests
- Updated ER diagram and data dictionary
- Setup and recreation steps
- Link: https://github.com/KarimaR23/HeyNeighbor.git

Neon

